Experiment 2: TF-IDF of a Corpus using MapReduce in Python

MapReduce Laboratory

Aryan Awasthi
Roll No.: 102315182
Subgroup: 402

1. Aim

To write a program in Python using the MapReduce programming model to compute the TF-IDF (Term Frequency, Inverse Document Frequency) of every word in every document of a corpus.

2. Theory

TF-IDF measures how important a word is to one document within a collection of documents (the corpus). A word scores high when it is frequent in that document but rare in the rest of the corpus. Search engines and text classifiers use it to rank and describe documents.

Quantity	Formula	Meaning
n	count of word w in document d	How often the word occurs in the document.
N	total words in document d	Length of the document.
tf(w, d)	n / N	Term frequency, normalised by document length.
D	number of documents	Size of the corpus.
df(w)	documents containing w	Document frequency.
idf(w)	log10(D / df)	Inverse document frequency. It is 0 for a word that appears in every document.
tfidf(w, d)	tf x idf	Final weight.

TF-IDF cannot be computed in one MapReduce job. N needs all the words of one document, and df needs all the documents of one word. Each of these is a different grouping key, and one job has only one shuffle. The computation is therefore a chain of **three jobs**:

Job	Mapper emits (key -> value)	Reducer emits	Computes
1	word@doc -> 1	word@doc -> n	term count
2	doc -> word,n	word@doc -> n,N	document length N
3	word -> doc,n,N	word@doc -> tf, idf, tfidf	df, idf and TF-IDF

The mapper of Job 1 learns the document name from the environment variable mapreduce_map_input_file, which Hadoop Streaming sets for every map task. D, the number of documents, is passed to Job 3 with -cmdenv TOTAL_DOCS=D.

3. Algorithm

1. **Job 1 map:** for each word in a line of document d, emit word@d TAB 1.
2. **Job 1 reduce:** add the 1s for each word@d to get n. Reducer 1 is also used as the combiner.
3. **Job 2 map:** turn word@d TAB n into d TAB word,n, so one reducer gets a whole document.
4. **Job 2 reduce:** add every n of the document to get N, then emit word@d TAB n,N.

5. **Job 3 map:** turn word@d TAB n,N into word TAB d,n,N, so one reducer gets every document containing the word.
6. **Job 3 reduce:** df = number of values received. For each document compute tf = n / N, idf = log10(D / df) and tfidf = tf x idf.

4. Program

mapper1.py

```
#!/usr/bin/env python3
"""Job 1 mapper: term count per document.
In : lines of one document (Hadoop sets the file name in mapreduce_map_input_file)
Out: word@doc \t 1
"""
import os
import re
import sys

path = os.environ.get("mapreduce_map_input_file") or os.environ.get("map_input_file",
"unknown")
doc = os.path.splitext(os.path.basename(path))[0]
for line in sys.stdin:
    for word in re.findall(r"[a-z0-9']+", line.lower()):
        print(f"{word}@{doc}\t1")
```

reducer1.py (also the combiner of Job 1)

```
#!/usr/bin/env python3
"""Job 1 reducer: n = number of times a word occurs in a document (also the combiner).
In : word@doc \t count   (sorted)
Out: word@doc \t n
"""
import sys

current, total = None, 0
for line in sys.stdin:
    key, count = line.rstrip("\n").split("\t", 1)
    if key != current:
        if current is not None:
            print(f"{current}\t{total}")
        current, total = key, 0
    total += int(count)
if current is not None:
    print(f"{current}\t{total}")
```

mapper2.py

```
#!/usr/bin/env python3
"""Job 2 mapper: re-key by document so one reducer sees every word of a document.
```

```
    In : word@doc \t n
    Out: doc \t word,n
    """
    import sys

    for line in sys.stdin:
        key, n = line.rstrip("\n").split("\t", 1)
        word, doc = key.rsplit("@", 1)
        print(f"{doc}\t{word},{n}")
```

reducer2.py

```
    #!/usr/bin/env python3
    """Job 2 reducer: N = total number of words in the document.
    In : doc \t word,n   (sorted by doc)
    Out: word@doc \t n,N
    """
    import sys


    def emit(doc, words):
        total = sum(n for _, n in words)
        for word, n in words:
            print(f"{word}@{doc}\t{n},{total}")


    current, words = None, []
    for line in sys.stdin:
        doc, value = line.rstrip("\n").split("\t", 1)
        word, n = value.rsplit(",", 1)
        if doc != current:
            if current is not None:
                emit(current, words)
            current, words = doc, []
        words.append((word, int(n)))
    if current is not None:
        emit(current, words)
```

mapper3.py

```
    #!/usr/bin/env python3
    """Job 3 mapper: re-key by word so one reducer sees every document that contains it.
    In : word@doc \t n,N
    Out: word \t doc,n,N
    """
    import sys

    for line in sys.stdin:
        key, value = line.rstrip("\n").split("\t", 1)
```

```
    word, doc = key.rsplit("@", 1)
    print(f"{word}\t{doc},{value}")
```

reducer3.py

```
#!/usr/bin/env python3
"""Job 3 reducer: df, IDF and TF-IDF.
In : word \t doc,n,N   (sorted by word)
Out: word@doc \t tf=..,idf=..,tfidf=..
  tf    = n / N              (share of the document's words that are this word)
  idf   = log10(D / df)      (D = documents in corpus, df = documents containing the word)
  tfidf = tf * idf
D is passed in by the driver: -cmdenv TOTAL_DOCS=<D>
"""
import math
import os
import sys

D = int(os.environ["TOTAL_DOCS"])


def emit(word, postings):
    idf = math.log10(D / len(postings))
    for doc, n, total in postings:
        tf = n / total
        print(f"{word}@{doc}\ttf={tf:.4f},idf={idf:.4f},tfidf={tf * idf:.4f}")


current, postings = None, []
for line in sys.stdin:
    word, value = line.rstrip("\n").split("\t", 1)
    doc, n, total = value.split(",")
    if word != current:
        if current is not None:
            emit(current, postings)
        current, postings = word, []
    postings.append((doc, int(n), int(total)))
if current is not None:
    emit(current, postings)
```

5. Execution

Locally (Windows, macOS or Linux, Python 3 only):

```
$ python run_demo.py
```

On Hadoop (run_hadoop.sh), three chained Hadoop Streaming jobs:

```
$ hdfs dfs -put data/corpus/*.txt /user/$USER/tfidf/input/
$ hadoop jar $STREAMING_JAR -files mapper1.py,reducer1.py -mapper "python3
mapper1.py" \
   -combiner "python3 reducer1.py" -reducer "python3 reducer1.py" \
   -input /user/$USER/tfidf/input -output /user/$USER/tfidf/job1
$ hadoop jar $STREAMING_JAR -files mapper2.py,reducer2.py -mapper "python3
mapper2.py" \
   -reducer "python3 reducer2.py" -input /user/$USER/tfidf/job1 -output
/user/$USER/tfidf/job2
$ hadoop jar $STREAMING_JAR -files mapper3.py,reducer3.py -cmdenv
TOTAL_DOCS=5 \
   -mapper "python3 mapper3.py" -reducer "python3 reducer3.py" \
   -input /user/$USER/tfidf/job2 -output /user/$USER/tfidf/job3
$ hdfs dfs -cat /user/$USER/tfidf/job3/part-*
```

6. Output

The output below is the real output of python run_demo.py. It shows the data after every job.

Input corpus

doc1.txt (3 lines)

```
$ cat data/corpus/doc1.txt
Hadoop stores big data in HDFS. HDFS splits every file into blocks and stores the blocks
on many machines. HDFS copies each block to three machines, so the blocks stay safe when
one of the machines fails.
```

doc2.txt (3 lines)

```
$ cat data/corpus/doc2.txt
MapReduce is the processing model of Hadoop. The mapper reads the data and emits key
value
pairs. The reducer receives every value of one key and combines the values. MapReduce
sends each key to one reducer.
```

doc3.txt (3 lines)

```
$ cat data/corpus/doc3.txt
Python is a simple programming language. Python programs can run on Hadoop through
Hadoop
Streaming. Streaming passes the data to the Python program as lines of text, and the
program prints its output as lines of text.
```

doc4.txt (3 lines)

```
$ cat data/corpus/doc4.txt
A search engine ranks documents for a query. TF IDF gives a high weight to a word that is
frequent in one document but rare in other documents. The engine returns the documents
with the highest weight for the query.
```

doc5.txt (2 lines)

```
$ cat data/corpus/doc5.txt
The cricket match started late because of rain. The team batted well and won the match in
the last over. The fans cheered for the team after the cricket match.
```

Job 1: term count n for every (word, document)

Mapper 1 output (word@doc TAB 1) (180 lines)

```
$ python mapper1.py < each document
hadoop@doc1    1
stores@doc1    1
big@doc1    1
data@doc1    1
in@doc1    1
hdfs@doc1    1
... (174 more lines)
```

Shuffle and Sort (180 lines)

```
$ ... | sort
a@doc3    1
a@doc4    1
a@doc4    1
a@doc4    1
a@doc4    1
after@doc5    1
... (174 more lines)
```

Reducer 1 output (word@doc TAB n) (125 lines)

```
$ ... | sort | python reducer1.py
a@doc3    1
a@doc4    4
after@doc5    1
and@doc1    1
and@doc2    2
and@doc3    1
... (119 more lines)
```

Job 2: document length N, giving n and N for every (word, document)

Mapper 2 output after Shuffle and Sort (doc TAB word,n) (125 lines)

```
$ python mapper2.py < job1 | sort
doc1    and,1
doc1    big,1
doc1    block,1
doc1    blocks,3
doc1    copies,1
```

```
doc1    data,1
... (119 more lines)
```

Reducer 2 output (word@doc TAB n,N) (125 lines)

```
$ ... | python reducer2.py
and@doc1    1,37
big@doc1    1,37
block@doc1    1,37
blocks@doc1    3,37
copies@doc1    1,37
data@doc1    1,37
... (119 more lines)
```

Job 3: document frequency df, IDF and TF-IDF (D = 5 documents)

Mapper 3 output after Shuffle and Sort (word TAB doc,n,N) (125 lines)

```
$ python mapper3.py < job2 | sort
a    doc3,1,37
a    doc4,4,40
after    doc5,1,30
and    doc1,1,37
and    doc2,2,36
and    doc3,1,37
... (119 more lines)
```

Reducer 3 output (word@doc TAB tf, idf, tfidf) (125 lines)

```
$ ... | TOTAL_DOCS=5 python reducer3.py
a@doc3    tf=0.0270,idf=0.3979,tfidf=0.0108
a@doc4    tf=0.1000,idf=0.3979,tfidf=0.0398
after@doc5    tf=0.0333,idf=0.6990,tfidf=0.0233
and@doc1    tf=0.0270,idf=0.0969,tfidf=0.0026
and@doc2    tf=0.0556,idf=0.0969,tfidf=0.0054
and@doc3    tf=0.0270,idf=0.0969,tfidf=0.0026
... (119 more lines)
```

Example: every line for 'the' (5 lines)

```
$ grep '^the@' output/tfidf.txt
the@doc1    tf=0.0811,idf=0.0000,tfidf=0.0000
the@doc2    tf=0.1389,idf=0.0000,tfidf=0.0000
the@doc3    tf=0.0811,idf=0.0000,tfidf=0.0000
the@doc4    tf=0.1000,idf=0.0000,tfidf=0.0000
the@doc5    tf=0.2333,idf=0.0000,tfidf=0.0000
```

Example: every line for 'hadoop' (3 lines)

```
$ grep '^hadoop@' output/tfidf.txt
hadoop@doc1    tf=0.0270,idf=0.2218,tfidf=0.0060
```

hadoop@doc2 tf=0.0278,idf=0.2218,tfidf=0.0062
hadoop@doc3 tf=0.0541,idf=0.2218,tfidf=0.0120

Result: top 3 terms of each document

Document	Rank	Word	tf	idf	TF-IDF
doc1	1	**blocks**	0.0811	0.6990	0.0567
doc1	2	**hdfs**	0.0811	0.6990	0.0567
doc1	3	**machines**	0.0811	0.6990	0.0567
doc2	1	**key**	0.0833	0.6990	0.0582
doc2	2	**mapreduce**	0.0556	0.6990	0.0388
doc2	3	**reducer**	0.0556	0.6990	0.0388
doc3	1	**python**	0.0811	0.6990	0.0567
doc3	2	**as**	0.0541	0.6990	0.0378
doc3	3	**lines**	0.0541	0.6990	0.0378
doc4	1	**documents**	0.0750	0.6990	0.0524
doc4	2	**a**	0.1000	0.3979	0.0398
doc4	3	**engine**	0.0500	0.6990	0.0349
doc5	1	**match**	0.1000	0.6990	0.0699
doc5	2	**cricket**	0.0667	0.6990	0.0466
doc5	3	**team**	0.0667	0.6990	0.0466

Summary and verification (5 lines)

```
$ python run_demo.py
Documents (D)                  : 5
Words in corpus (mapper 1 records)  : 180
(word, document) pairs         : 125
Distinct words                 : 97
Matches a direct Python computation: YES
```

7. Result

TF-IDF was computed for all 125 (word, document) pairs of a 5-document corpus with three chained MapReduce jobs written in Python. The values match a direct computation done without MapReduce. Words that occur in every document, such as “the”, get idf = 0 and so TF-IDF = 0. The highest scores go to words that describe each document: “blocks” and “hdfs” (storage), “key” and “mapreduce” (processing), “python”, “documents” and “match” (cricket). A few common words such as “a” and “as” still rank high because the documents are short. Removing stop words in the first mapper would filter them out.